

GDB 双重调试系统调用实验报告

实验目标

使用"双重 GDB"方案，从 QEMU 模拟器层面观察系统调用指令的处理：

- 观察 `ecall` 指令如何被 QEMU 捕获和处理
- 观察 `sret` 指令如何实现特权级切换
- 理解 TCG (Tiny Code Generator) 指令翻译机制

实验环境

组件	版本/说明
QEMU	qemu-4.1.1 (从源码编译，带调试信息)
x86_64 GDB	调试 QEMU 进程本身
riscv64 GDB	调试 ucore 内核和用户程序
OS	ucore Lab5

需要修改 Makefile 使用带调试信息的 QEMU：

```
QEMU := /root/Downloads/qemu-4.1.1/riscv64-sofmmu/qemu-system-riscv64
```

1 实验过程

本实验使用三个终端协同工作：

- 终端1**：运行 QEMU（执行 ucore）
- 终端2**：x86_64 GDB 调试 QEMU 进程本身
- 终端3**：riscv64 GDB 调试 ucore 内核和用户程序

1.1 预备阶段

终端1：启动 QEMU

```
make debug
```

终端2：附加到 QEMU 进程

```
pgrep -f qemu-system-riscv64  # 获取 QEMU 的 PID
sudo gdb
(gdb) attach <PID>
(gdb) c                        # 让 QEMU 继续运行
```

终端3：连接并加载用户程序符号

```
make gdb
(gdb) set remotetimeout unlimited
(gdb) add-symbol-file obj/__user_exit.out # 加载用户程序符号
```

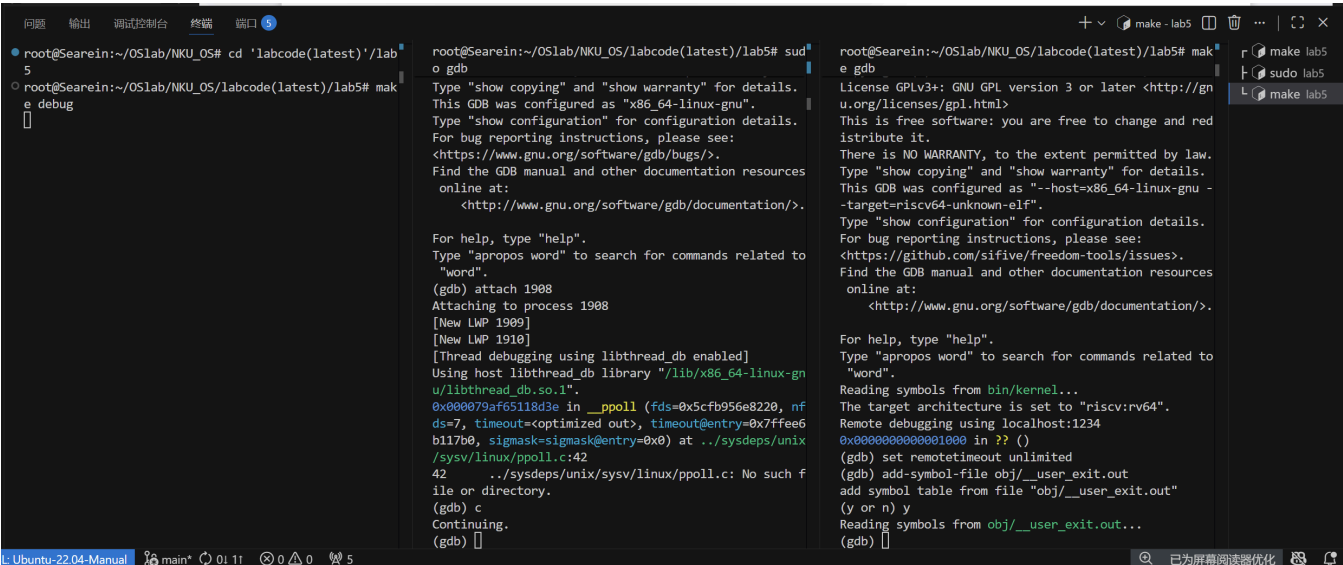


图1说明：三个终端的初始状态。左侧终端1执行 `make debug` 启动 QEMU；中间终端2通过 `pgrep` 获取 QEMU 进程 PID 后使用 `attach` 命令附加到该进程；右侧终端3执行 `make gdb` 连接到 QEMU 的 GDB server，并加载用户程序符号文件 `obj/__user_exit.out`。

1.2 调试 ecall 指令

步骤1：定位 ecall 指令

在终端3中，使用 `si` 单步执行并用 `x/8i $pc` 查看当前指令：

```
(gdb) si
(gdb) si
(gdb) x/8i $pc
```

```
root@Searein:~/OSlab/NKU_OS/labcode(latest)/lab5# r
e gdb
```

```
0x800104 <syscall+44>:      ecall
0x800108 <syscall+48>:      sd      a0,28(sp)
0x80010c <syscall+52>:      lw      a0,28(sp)
0x80010e <syscall+54>:      addi     sp,sp,144
0x800110 <syscall+56>:      ret
0x800112 <sys_exit>: mv      a1,a0
(gdb) si
0x0000000000800102      19      asm volatile (
(gdb) si
0x0000000000800104      19      asm volatile (
(gdb) x/8i $pc
=> 0x800104 <syscall+44>:      ecall
0x800108 <syscall+48>:      sd      a0,28(sp)
0x80010c <syscall+52>:      lw      a0,28(sp)
0x80010e <syscall+54>:      addi     sp,sp,144
0x800110 <syscall+56>:      ret
0x800112 <sys_exit>: mv      a1,a0
0x800114 <sys_exit+2>:      li      a0,1
0x800116 <sys_exit+4>:
      j      0x8000d8 <syscall>
(gdb) █
```

图2说明：在终端3中通过单步执行（`si`）和反汇编（`x/8i $pc`）定位到 `ecall` 指令。可以看到：

- 地址 `0x800104 <syscall+44>` 处即为 `ecall` 指令
- 当前 PC 指向 `ecall`，准备执行系统调用
- 后续指令包括 `sd a0,28(sp)`（保存返回值）、`ret`（返回）等

步骤2：在 QEMU 中设置断点

在终端2中暂停 QEMU 并设置断点：

```
(gdb) Ctrl+C          # 暂停 QEMU
(gdb) b riscv_cpu_do_interrupt # 在中断处理函数设断点
(gdb) c
```

步骤3：触发 ecall 并观察

在终端3中执行 `si` 触发 ecall，终端2命中断点：

```
# 终端3
(gdb) si                                # 执行 ecall

# 终端2
(gdb) layout src                        # 进入源码观察模式
```

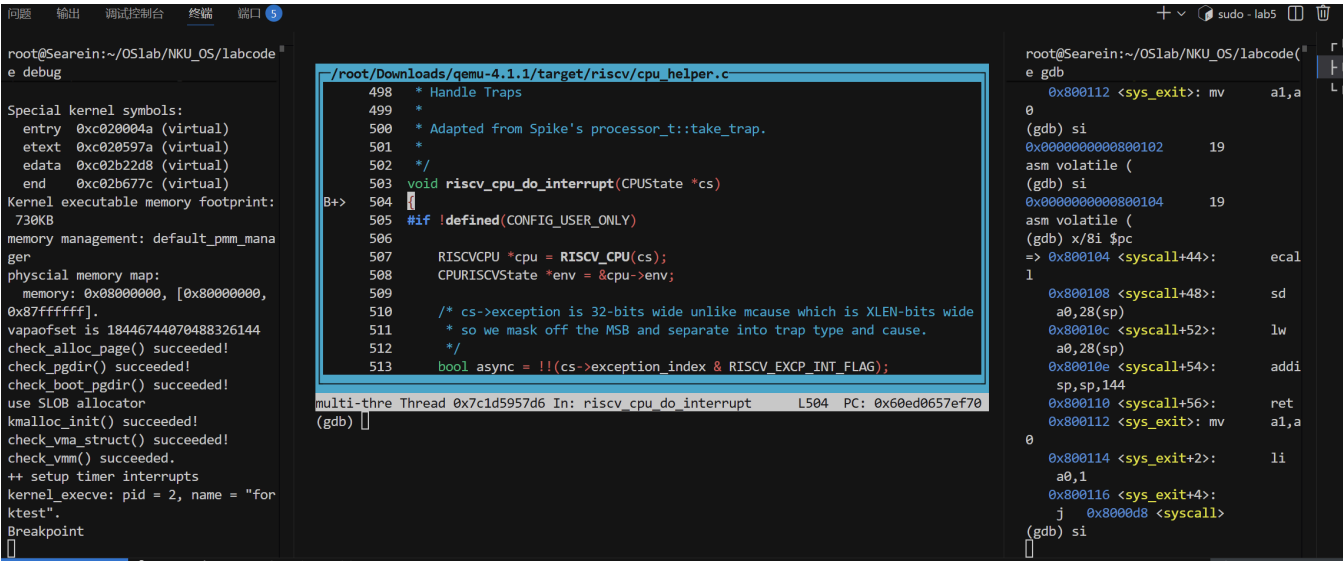


图3说明：当终端3执行 `si` 触发 `ecall` 后，终端2的 x86_64 GDB 命中 `riscv_cpu_do_interrupt` 断点。使用 `layout src` 进入源码观察模式，可以看到 QEMU 源码文件 `target/riscv/cpu_helper.c`，当前停在函数入口处。右侧终端3显示程序正在等待（因为 QEMU 被暂停）。

步骤4：观察关键变量

在终端2中查看异常类型：

```
(gdb) list
(gdb) p cs->exception_index
$1 = 8
(gdb) n                                # 单步观察处理流程
```

```
/root/Downloads/qemu-4.1.1/target/riscv/cpu_helper.c
535     case RISC_V_EXCP_LOAD_PAGE_FAULT:
536     case RISC_V_EXCP_STORE_PAGE_FAULT:
537         tval = env->badaddr;
538         break;
539     default:
540         break;
541 }
542 /* ecall is dispatched as one cause so translate based on mode */
543 if (cause == RISC_V_EXCP_U_ECALL) {
> 544     assert(env->priv <= 3);
545     cause = ecall_cause_map[env->priv];
546 }
547 }
548
549 trace_riscv_trap(env->mhartid, async, cause, env->pc, tval, cause < 16
550                 (async ? riscv_intr_names : riscv_excp_names)[cause] : "(unknown)")
551
552 if (env->priv <= PRV_S &&
```

```
multi-thre Thread 0x7c1d5957d6 In: riscv_cpu_do_interrupt      L544  PC: 0x60ed0657f00c
(gdb) list
(gdb) p cs->exception_index
$1 = 8
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
```

图4说明：在 `riscv_cpu_do_interrupt` 函数中查看 `cs->exception_index` 的值为 **8**，这表示 **U 态 ecall**（`RISC_V_EXCP_U_ECALL`）。源码显示当前执行到第544行的 `assert(env->priv <= 3)`，正在处理 ecall 异常类型的映射。

步骤5：观察 S 态处理流程

继续单步执行，观察如何切换到 S 态：

```

552     if (env->priv <= PRV_S &&
553         cause < TARGET_LONG_BITS && ((deleg >> cause) & 1)) {
554         /* handle the trap in S-mode */
555         target_ulong s = env->mstatus;
556         s = set_field(s, MSTATUS_SPIE, env->priv_ver >= PRIV_VERSION_1_10_0
557             get_field(s, MSTATUS_SIE) : get_field(s, MSTATUS_UIE << env->pr
558         s = set_field(s, MSTATUS_SPP, env->priv);
559         s = set_field(s, MSTATUS_SIE, 0);
560         env->mstatus = s;
561         env->scause = cause | ((target_ulong)async << (TARGET_LONG_BITS - 1
562         env->sepc = env->pc;
563         env->sbadaddr = tval;
564         env->pc = (env->stvec >> 2 << 2) +
565             ((async && (env->stvec & 3) == 1) ? cause * 4 : 0);
566         riscv_cpu_set_mode(env, PRV_S);
567     } else {
568         /* handle the trap in M-mode */
569         target_ulong s = env->mstatus;

```

thre Thread 0x7c1d5957d6 In: riscv_cpu_do_interrupt L566 PC: 0x60ed0657f10e

n
n
n
n
n
n
n
n
n
n
n

图5说明：继续单步执行（多次 `n`）后，代码执行到第566行 `riscv_cpu_set_mode(env, PRV_S)`。这是 `ecall` 处理的关键步骤：

- 第555-560行：保存和设置 `mstatus` 寄存器（SPIE、SPP、SIE 字段）
- 第561行：设置 `scause = cause`（异常原因）
- 第562行：设置 `sepc = pc`（保存返回地址）
- 第564-565行：设置 `pc = stvec`（跳转到中断向量）
- 第566行：切换到 S 态（Supervisor Mode）

1.3 调试 sret 指令

步骤1：设置 helper_sret 断点

在终端2中删除旧断点，设置新断点：

```
(gdb) delete # 删除旧断点
(gdb) b helper_sret # 在 sret 处理函数设断点
(gdb) c
```

步骤2：继续执行并观察

在终端3中继续执行，等待命中 `helper_sret`：

```
# 终端3
(gdb) c

# 终端2 命中断点后
(gdb) p/x env->sepc # 查看返回地址
```

The screenshot shows a GDB debugging session with two terminals. The left terminal (Terminal 2) shows the `riscv_cpu_do_interrupt` function in `target/riscv/cpu_helper.c`. The function is at line 552, and the breakpoint is at line 566. The right terminal (Terminal 3) shows the user program's execution, which is currently at the `syscall` instruction at address `0x800104`. The program is in the `syscall` function, and the next instruction is `ecall` at address `0x800108`.

```

/root/Downloads/qemu-4.1.1/target/riscv/cpu_helper.c
552     if (env->priv <= PRV_S &&
553         cause < TARGET_LONG_BITS && ((deleg >> cause) & 1)) {
554         /* handle the trap in S-mode */
555         target_ulong s = env->mstatus;
556         s = set_field(s, MSTATUS_SPIE, env->priv_ver >= PRIV_VERSION_1_10_0);
557         get_field(s, MSTATUS_SIE) : get_field(s, MSTATUS_UIE << env->priv);
558         s = set_field(s, MSTATUS_SPP, env->priv);
559         s = set_field(s, MSTATUS_SIE, 0);
560         env->mstatus = s;
561         env->scause = cause | ((target_ulong)async << (TARGET_LONG_BITS - 1));
562         env->sepc = env->pc;
563         env->sbadaddr = tval;
564         env->pc = (env->stvec >> 2 << 2) +
565                 ((async && (env->stvec & 3) == 1) ? cause * 4 : 0);
> 566         riscv_cpu_set_mode(env, PRV_S);
567     } else {
568         /* handle the trap in M-mode */
569         target_ulong s = env->mstatus;

multi-thre Thread 0x7c1d5957d6 In: riscv_cpu_do_interrupt L566 PC: 0x60ed0657f10e
(gdb) n
(gdb) n
(gdb) p/x env->sepc
$2 = 0x800104
(gdb) p/x env->mstatus
$3 = 0x80000000000046020
(gdb) p/x env->scause
$4 = 0x8
(gdb) p/x env->sbadaddr
$5 = 0x0
(gdb)

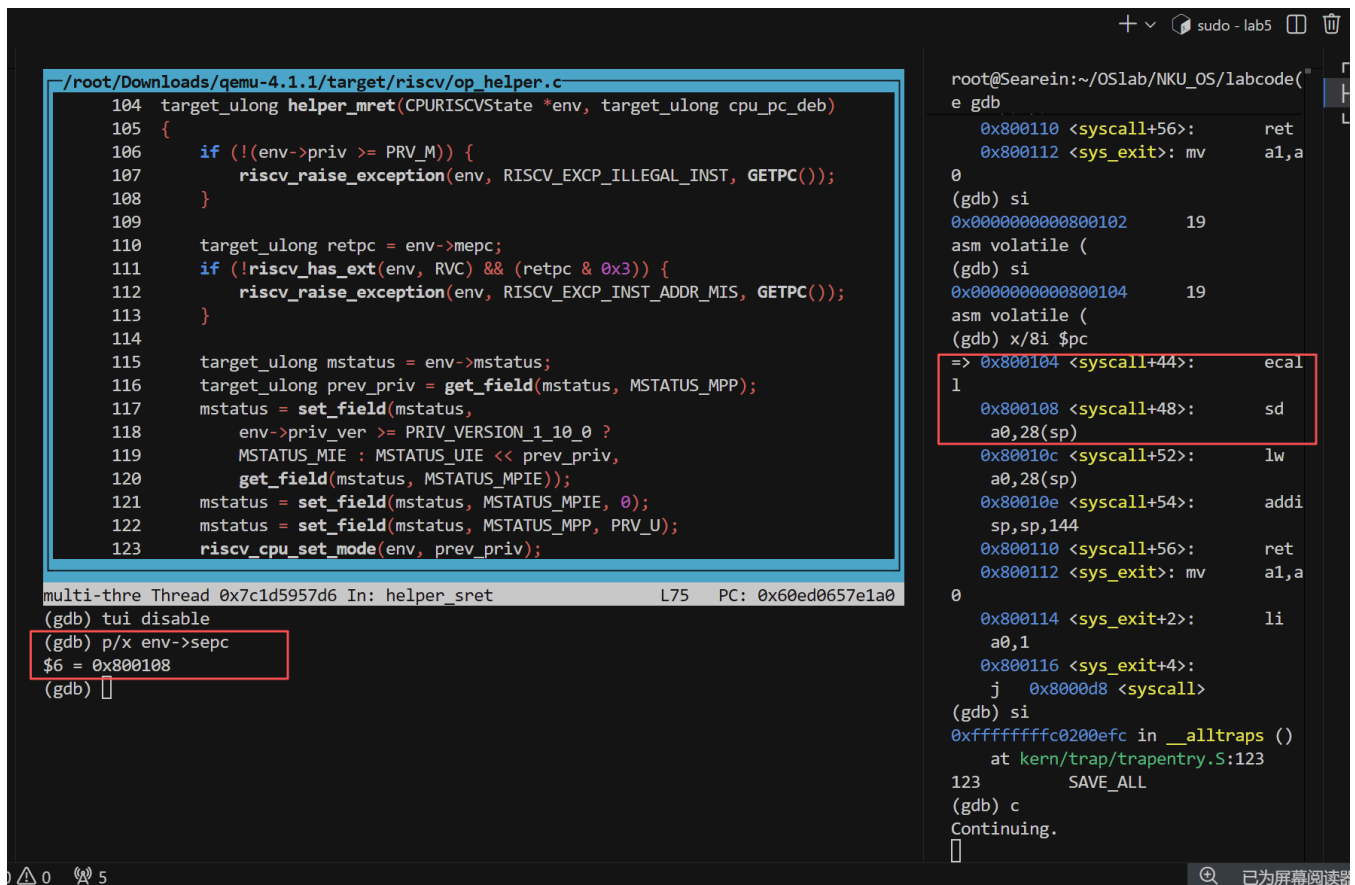
root@Searein:~/OSlab/NKU_OS/labcode("
e gdb
a0,28(sp)
0x80010e <syscall+54>: addi
sp,sp,144
0x800110 <syscall+56>: ret
0x800112 <sys_exit>: mv
a1,a
0
(gdb) si
0x0000000000000102 19
asm volatile (
(gdb) si
0x0000000000000104 19
asm volatile (
(gdb) x/8i $pc
=> 0x800104 <syscall+44>: ecal
1
0x800108 <syscall+48>: sd
a0,28(sp)
0x80010c <syscall+52>: lw
a0,28(sp)
0x80010e <syscall+54>: addi
sp,sp,144
0x800110 <syscall+56>: ret
0x800112 <sys_exit>: mv
a1,a
0
0x800114 <sys_exit+2>: li
a0,1
0x800116 <sys_exit+4>: j
0x800d8 <syscall>
(gdb) si

```

图6说明：命中 `helper_sret` 断点后，可以观察到：

- 当前在 `target/riscv/cpu_helper.c` 的 `riscv_cpu_do_interrupt` 函数附近
- 右侧终端3显示用户程序的 `syscall` 调用链
- 通过 `p/x env->sepc` 可以查看即将返回的地址（如 `0x800108`，即 `ecall` 的下一条指令）

步骤3：观察 helper_sret 内部逻辑



```
/root/Downloads/qemu-4.1.1/target/riscv/op_helper.c
104 target_ulong helper_mret(CPURISCVState *env, target_ulong cpu_pc_deb)
105 {
106     if (!(env->priv >= PRV_M)) {
107         riscv_raise_exception(env, RISCVC_EXCP_ILLEGAL_INST, GETPC());
108     }
109
110     target_ulong retpc = env->mepc;
111     if (!riscv_has_ext(env, RVC) && (retpc & 0x3)) {
112         riscv_raise_exception(env, RISCVC_EXCP_INST_ADDR_MIS, GETPC());
113     }
114
115     target_ulong mstatus = env->mstatus;
116     target_ulong prev_priv = get_field(mstatus, MSTATUS_MPP);
117     mstatus = set_field(mstatus,
118         env->priv_ver >= PRIV_VERSION_1_10_0 ?
119         MSTATUS_MIE : MSTATUS_UIE << prev_priv,
120         get_field(mstatus, MSTATUS_MPIE));
121     mstatus = set_field(mstatus, MSTATUS_MPIE, 0);
122     mstatus = set_field(mstatus, MSTATUS_MPP, PRV_U);
123     riscv_cpu_set_mode(env, prev_priv);
124 }
```

```
root@Searein:~/OSlab/NKU_OS/labcode(
e gdb
0x800110 <syscall+56>: ret
0x800112 <sys_exit>: mv a1,a
0
(gdb) si
0x0000000000000102 19
asm volatile (
(gdb) si
0x0000000000000104 19
asm volatile (
(gdb) x/8i $pc
=> 0x800104 <syscall+44>: ecal
1
0x800108 <syscall+48>: sd
a0,28(sp)
0x80010c <syscall+52>: lw
a0,28(sp)
0x80010e <syscall+54>: addi
sp,sp,144
0x800110 <syscall+56>: ret
0x800112 <sys_exit>: mv a1,a
0
0x800114 <sys_exit+2>: li
a0,1
0x800116 <sys_exit+4>: j 0x8000d8 <syscall>
(gdb) si
0xfffffffffc0200efc in __alltraps ()
at kern/trap/trapentry.S:123
123 SAVE_ALL
(gdb) c
Continuing.
```

图7说明：进入 helper_sret 函数内部（target/riscv/op_helper.c）。可以看到：

- 第80行：retpc = env->sepc 获取返回地址
- 第90-97行：恢复 mstatus 寄存器状态
- 第98行：riscv_cpu_set_mode(env, prev_priv) 切换回之前的特权级（U 态）
- 使用 p/x env->sepc 查看返回地址为 0x800108

2 代码解释

2.1 riscv_cpu_do_interrupt 函数

该函数位于 target/riscv/cpu_helper.c，负责处理 RISC-V CPU 的中断和异常，模拟硬件的异常处理行为。

```
void riscv_cpu_do_interrupt(CPUState *cs)
{
    #if !defined(CONFIG_USER_ONLY)
        RISCVCPU *cpu = RISCVCPU(cs);
        CPURISCVState *env = &cpu->env;

        /* 从 exception_index 中提取异常类型 */
        bool async = !(cs->exception_index & RISCVC_EXCP_INT_FLAG);
        target_ulong cause = cs->exception_index & RISCVC_EXCP_INT_MASK;
        target_ulong deleg = async ? env->mideleg : env->medeleg;
```

```
target_ulong tval = 0;

/* ecall 异常类型映射表 */
static const int ecall_cause_map[] = {
    [PRV_U] = RISC_V_EXCP_U_ECALL, // U态ecall -> 8
    [PRV_S] = RISC_V_EXCP_S_ECALL, // S态ecall -> 9
    [PRV_H] = RISC_V_EXCP_H_ECALL, // H态ecall -> 10
    [PRV_M] = RISC_V_EXCP_M_ECALL  // M态ecall -> 11
};
```

关键流程分析

1. 异常类型判断

```
if (!async) {
    /* 对于同步异常，设置 tval (trap value) */
    switch (cause) {
        case RISC_V_EXCP_INST_PAGE_FAULT:
        case RISC_V_EXCP_LOAD_PAGE_FAULT:
        case RISC_V_EXCP_STORE_PAGE_FAULT:
            tval = env->badaddr; // 记录出错地址
            break;
        default:
            break;
    }
    /* ecall 根据当前特权级映射到具体的异常号 */
    if (cause == RISC_V_EXCP_U_ECALL) {
        assert(env->priv <= 3);
        cause = ecall_cause_map[env->priv];
    }
}
```

2. S 态异常处理（核心逻辑）

当异常被委托给 S 态处理时（通过 `medeleg` 寄存器配置）：

```
if (env->priv <= PRV_S &&
    cause < TARGET_LONG_BITS && ((deleg >> cause) & 1)) {
    /* 在 S 态处理异常 */
    target_ulong s = env->mstatus;

    // 保存当前中断使能状态到 SPIE
    s = set_field(s, MSTATUS_SPIE, get_field(s, MSTATUS_SIE));
    // 保存当前特权级到 SPP
    s = set_field(s, MSTATUS_SPP, env->priv);
    // 关闭中断
    s = set_field(s, MSTATUS_SIE, 0);
    env->mstatus = s;

    // 设置异常原因
    env->scause = cause | ((target_ulong)async << (TARGET_LONG_BITS - 1));
    // 保存返回地址（当前 PC）
```

```

env->sepc = env->pc;
// 保存错误地址
env->sbadaddr = tval;
// 跳转到中断向量
env->pc = (env->stvec >> 2 << 2) +
    ((async && (env->stvec & 3) == 1) ? cause * 4 : 0);
// 切换到 S 态
riscv_cpu_set_mode(env, PRV_S);
}

```

关键寄存器操作总结

操作	说明
<code>SPIE ← SIE</code>	保存进入异常前的中断使能状态
<code>SPP ← priv</code>	保存进入异常前的特权级（0=U态）
<code>SIE ← 0</code>	关闭中断，防止嵌套
<code>scause ← cause</code>	记录异常原因（8=U态ecall）
<code>sepc ← pc</code>	保存返回地址
<code>pc ← stvec</code>	跳转到 trap handler
<code>priv ← S</code>	切换到 Supervisor 态

2.2 helper_sret 函数

该函数位于 `target/riscv/op_helper.c`，负责处理 `sret` 指令，实现从 S 态返回到之前的特权级。

```

target_ulong helper_sret(CPURISCVState *env, target_ulong cpu_pc_deb)
{
    /* 特权级检查：必须在 S 态或更高特权级 */
    if (!(env->priv >= PRV_S)) {
        riscv_raise_exception(env, RISCV_EXCP_ILLEGAL_INST, GETPC());
    }

    /* 获取返回地址 */
    target_ulong retpc = env->sepc;

    /* 地址对齐检查（如果没有 C 扩展） */
    if (!riscv_has_ext(env, RVC) && (retpc & 0x3)) {
        riscv_raise_exception(env, RISCV_EXCP_INST_ADDR_MIS, GETPC());
    }

    /* TSR (Trap SRET) 检查 */
    if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
        get_field(env->mstatus, MSTATUS_TSR)) {
        riscv_raise_exception(env, RISCV_EXCP_ILLEGAL_INST, GETPC());
    }
}

```

```
}
```

核心逻辑：恢复状态

```
target_ulong mstatus = env->mstatus;

/* 获取之前保存的特权级 */
target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP);

/* 恢复中断使能状态: SIE ← SPIE */
mstatus = set_field(mstatus, MSTATUS_SIE, get_field(mstatus, MSTATUS_SPIE));

/* 清除 SPIE */
mstatus = set_field(mstatus, MSTATUS_SPIE, 0);

/* 将 SPP 设为 U 态（最低特权级） */
mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);

/* 切换回之前的特权级 */
riscv_cpu_set_mode(env, prev_priv);

env->mstatus = mstatus;

return retpc; // 返回到 sepc 保存的地址
}
```

关键寄存器操作总结

操作	说明
<code>prev_priv ← SPP</code>	读取之前保存的特权级
<code>SIE ← SPIE</code>	恢复中断使能状态
<code>SPIE ← 0</code>	清除 SPIE
<code>SPP ← U</code>	将 SPP 重置为 U 态
<code>priv ← prev_priv</code>	切换回之前的特权级
<code>pc ← sepc</code>	返回到保存的地址

2.3 sstatus 寄存器详解

`sstatus` (Supervisor Status) 寄存器是系统调用的"状态保存箱", 其关键字段:

`sstatus` 寄存器布局（关键字段）：

...	SPP	SPIE	SIE
	[8]	[5]	[1]

字段	位置	含义
SIE	bit[1]	Supervisor Interrupt Enable, S 态中断使能
SPIE	bit[5]	Supervisor Previous Interrupt Enable, 保存进入 S 态前的 SIE
SPP	bit[8]	Supervisor Previous Privilege, 保存进入 S 态前的特权级

ecall/sret 时的状态变化

`ecall` 时：

SPP	← 当前特权级 (U=0)	
SPIE	← SIE (保存中断状态)	
SIE	← 0 (关闭中断)	

`sret` 时：

特权级	← SPP (恢复到 U 态)	
SIE	← SPIE (恢复中断状态)	
SPIE	← 0	
SPP	← U (重置为最低特权)	

3 TCG Translation

3.1 什么是 TCG

TCG (Tiny Code Generator) 是 QEMU 的**动态二进制翻译引擎**，负责将客户机指令（如 RISC-V）翻译为宿主机指令（如 x86_64）执行。



核心概念

概念	说明
翻译块 (TB)	Translation Block，以基本块为单位进行翻译
code_gen_buffer	存放翻译后机器码的缓冲区
TCG IR	中间表示，先将客户机指令翻译为 IR，再翻译为宿主机指令

3.2 普通指令 vs 特权指令

TCG 对不同类型的指令采用不同的处理策略：

普通指令（如 add、ld、sd）

直接翻译为宿主机指令，在 code_gen_buffer 中高效执行：

```
RISC-V: add a0, a1, a2
      ↓ TCG 翻译
x86_64: lea eax, [rbx + rcx]  （示意）
```

特权指令（如 ecall、sret）

不直接翻译，而是生成"退出翻译块"的代码，回到 QEMU 主循环，由 C 语言 helper 函数处理：

```
1. TCG 遇到 ecall 指令
2. 生成代码：设置 exception_index = 8，退出 TB
3. 回到 QEMU 主循环
4. 主循环调用 riscv_cpu_do_interrupt()
5. C 函数模拟硬件行为（设置 sepc、scause、切换特权级）
6. 翻译新的 TB（从 stvec 开始），继续执行
```

3.3 TCG 执行流程

```
QEMU 主循环
while (true) {
    tb = tb_find(pc);           // 查找或翻译 TB
    cpu_exec(tb);               // 执行翻译后的代码
    if (exit_reason == EXCP)    // 遇到异常/特权指令
        cpu_handle_exception(); // 调用 C handler
}
```

ecall 的 TCG 处理

```
// target/riscv/translate.c 中 ecall 的翻译
static bool trans_ecall(DisasContext *ctx, arg_ecall *a)
{
    // 不生成实际计算指令，而是：
    // 1. 设置 exception_index
    generate_exception(ctx, RISCV_EXCP_U_ECALL);
    // 2. 结束当前翻译块
    ctx->base.is_jmp = DISAS_NORETURN;
    return true;
}
```

sret 的 TCG 处理

```
// target/riscv/translate.c 中 sret 的翻译
static bool trans_sret(DisasContext *ctx, arg_sret *a)
{
    // 生成调用 helper_sret 的代码
    gen_helper_sret(cpu_pc, cpu_env, cpu_pc);
    // 结束当前翻译块（因为 PC 会改变）
    ctx->base.is_jmp = DISAS_NORETURN;
    return true;
}
```

3.4 调试中观察到的 TCG 现象

在 `helper_sret` 函数中执行 `finish` 后，GDB 显示：

```
0x00007bbe6539848f in code_gen_buffer ()
No Source Available
```

这正是 TCG 工作的体现：

- `helper_sret` 返回后，控制权回到 `code_gen_buffer` 中的翻译代码
- 这段代码是**动态生成**的机器码，没有源码对应
- GDB 无法显示源码，只能显示地址

3.5 与 Lab2 页表调试的关联

Lab2 的双重 GDB 调试地址翻译时，同样涉及 TCG 机制：

实验	触发条件	QEMU 处理函数	TCG 行为
Lab2	访存 TLB 未命中	<code>get_physical_address()</code>	退出 TB，软件 walk 页表
Lab5	ecall 指令	<code>riscv_cpu_do_interrupt()</code>	退出 TB，模拟异常处理
Lab5	sret 指令	<code>helper_sret()</code>	调用 helper，返回后翻译新 TB

共同点：

1. 都需要"退出翻译块"回到 QEMU 主循环
2. 都由 C 函数模拟硬件行为
3. 处理完成后都需要翻译新的 TB 继续执行

本质理解：QEMU 通过 TCG 实现了"软件模拟硬件"——对于简单指令直接翻译执行，对于复杂的硬件行为（异常处理、页表查找、特权级切换）则用 C 代码精确模拟。

4 实验回顾

4.1 大模型交流

问题1：断点行号的确认

情景：看到指导书说在 syscall 函数设断点，但不确定具体应该在第几行。

交互过程：

- **我：**指导书说在 syscall 函数设断点，但是 syscall 函数有好几行，应该在哪一行设？
- **大模型：**建议在 `asm volatile (` 所在行（第19行）设断点，这是内联汇编的起始位置，可以在这里单步执行找到 `ecall` 指令。
- **结果：**按建议操作，成功定位到 `ecall`。

问题2：理解 `exception_index` 的含义

情景：在 `riscv_cpu_do_interrupt` 中用 `p cs->exception_index` 看到值为 8，不理解这个数字的含义。

交互过程：

- **我：**`cs->exception_index` 是 8，这是什么意思？
- **大模型：**解释了 `8 = RISC_V_EXCP_U_ECALL`，表示用户态的 `ecall` 异常。RISC-V 中不同特权级的 `ecall` 有不同的异常号：U 态是 8，S 态是 9，M 态是 11。
- **收获：**理解了异常号的编码规则。

问题3：理解 TCG 指令翻译

情景：在 `helper_sret` 执行 `finish` 后，GDB 跳到了 `code_gen_buffer`，显示 "No Source Available"，不知道发生了什么。

交互过程：

- **我：**为什么 `finish` 之后跳到了一个没有源码的地方？`code_gen_buffer` 是什么？
- **大模型：**解释了 TCG 动态二进制翻译的概念——QEMU 将 RISC-V 指令翻译为 x86_64 指令存放在 `code_gen_buffer` 中，这些是运行时生成的机器码，所以没有源码对应。
- **收获：**深入理解了 QEMU 的工作原理，明白了为什么叫"模拟器"。

问题4：理解 sstatus 寄存器

情景：在讨论 sret 的实现时，代码中频繁操作 `mstatus / sstatus`，不太理解这个寄存器的作用。

交互过程：

- **我**：sstatus 寄存器是干什么的？为什么 ecall 和 sret 都要操作它？
- **大模型**：解释了 sstatus 是"状态保存箱"，其中 SPP 保存之前的特权级，SPIE 保存之前的中断状态，SIE 是当前中断使能。ecall 时保存状态，sret 时恢复状态。
- **收获**：理解了特权级切换时状态保存/恢复的机制。

问题5：env->priv 为什么还是 1

情景：在 `helper_sret` 函数中查看 `env->priv`，期望看到切换后的值（0=U态），但看到的还是 1（S态）。

交互过程：

- **我**：为什么在 `helper_sret` 里面 `env->priv` 还是 1？不是应该切换到 U 态了吗？
- **大模型**：`helper_sret` 函数还在执行中，`riscv_cpu_set_mode()` 是在函数末尾才被调用的。需要执行到函数返回，特权级才会真正切换。
- **收获**：理解了调试时观察变量的时机很重要，函数执行过程中变量的值会逐步变化。

4.2 问题与细节

细节1：用户程序符号加载

现象：在 riscv64 GDB 中设置断点 `b user/libs/syscall.c:19` 时报错：

```
Cannot access memory at address 0x8000f8
```

原因：用户程序尚未被加载到内存，虚拟地址 `0x8000f8` 还不可访问。

解决：GDB 提示是否设为 pending breakpoint，选择 `y`，等程序加载后断点自动生效。

细节2：变量被优化

现象：在 `riscv_cpu_do_interrupt` 中查看某些变量时显示 `<optimized out>`：

```
(gdb) p/x env->sepc
$1 = <optimized out>
```

原因：QEMU 编译时开启了优化（-O2），部分局部变量被优化掉或存放在寄存器中。

解决方案：

1. 单步执行到变量被使用的位置再查看
2. 查看结构体成员（如 `p_cs->exception_index`）而非局部变量
3. 重新编译 QEMU 时使用 `-O0` 禁用优化（但会很慢）

细节3：finish 后跳入 code_gen_buffer

现象：在 `helper_sret` 中执行 `finish` 后，GDB 显示：

```
0x00007bbe6539848f in code_gen_buffer ()
No Source Available
```

理解：这不是错误，而是 TCG 正常工作的体现。`helper_sret` 返回后，执行流回到动态生成的翻译代码中，继续执行用户程序。

细节4：n 和 s 命令的区别

现象：在 x86 GDB 中用 `n` 单步时，一下跳过了很多代码。

原因：

- `n` (next)：执行到下一行源代码，**跳过**函数调用
- `s` (step)：单步执行，**进入**函数调用
- `si` (step instruction)：单步执行一条机器指令

建议：观察函数内部逻辑时用 `s` 或 `n`，观察底层行为时用 `si`。

细节5：三终端同步问题

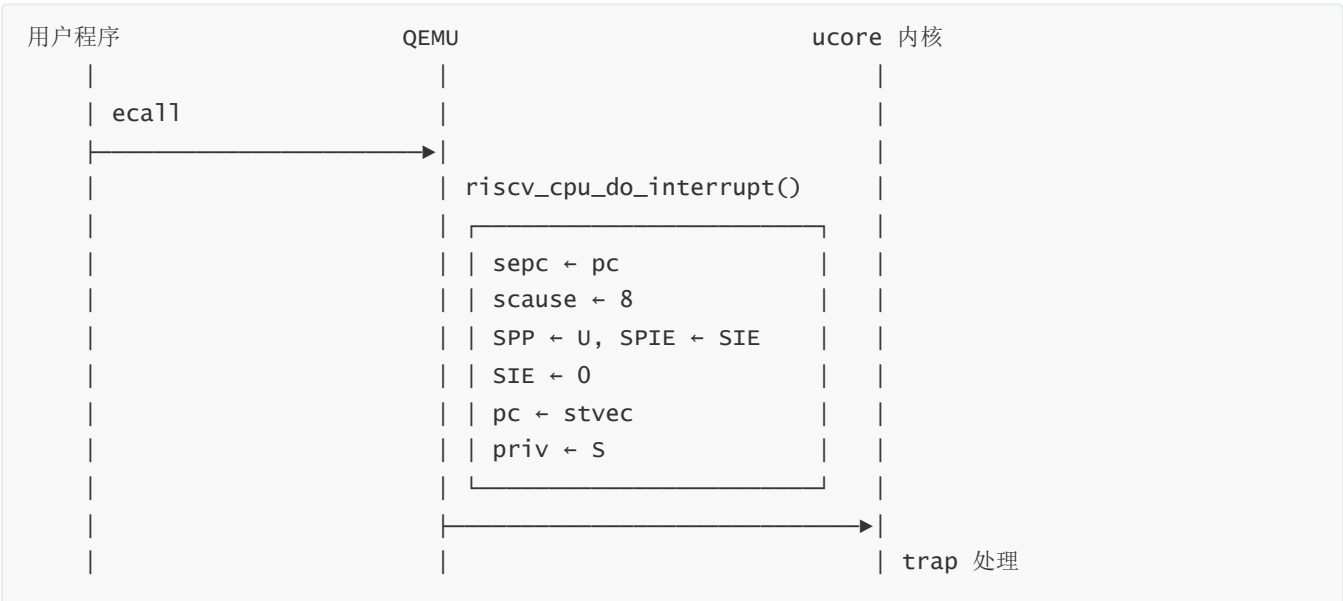
现象：终端2暂停 QEMU 后，终端3 的 GDB 也会卡住；终端2 继续执行后，终端3 才能响应。

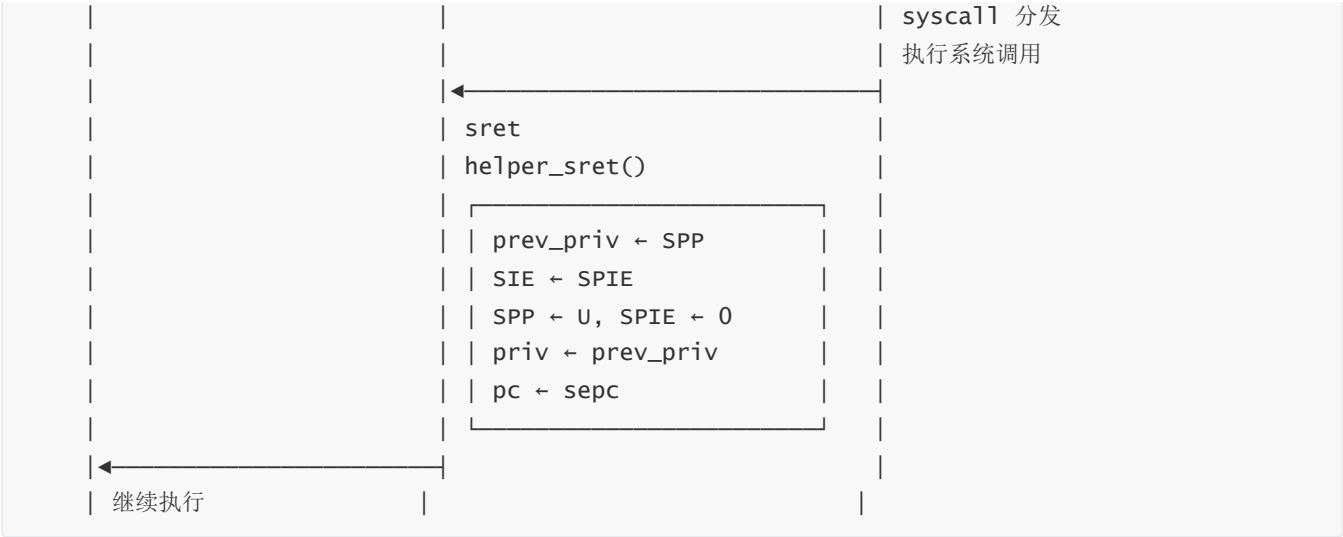
理解：QEMU 是单进程，x86 GDB 暂停它后，整个模拟器停止，包括 GDB server。riscv64 GDB 通过 GDB server 与 QEMU 通信，所以也会被阻塞。

5 实验总结

5.1 系统调用的完整流程

通过本次双重 GDB 调试实验，完整观察了系统调用的底层实现：





5.2 核心收获

1. 理解 QEMU 如何模拟硬件

QEMU 不是简单地"解释执行"RISC-V 指令，而是通过 TCG 将指令翻译为 x86_64 机器码高效执行。对于特权指令（ecall、sret），则用 C 函数精确模拟硬件行为。

2. 理解特权级切换机制

- **ecall**: 保存状态（sepc、scause、sstatus），关中断，跳转到 trap handler，切换到 S 态
- **sret**: 恢复状态，恢复中断，切换回 U 态，返回到用户程序

3. 理解双重调试的价值

调试层面	工具	观察内容
模拟器层	x86_64 GDB	QEMU 如何处理指令、TCG 翻译、helper 函数
被模拟系统层	riscv64 GDB	ucore 内核和用户程序的执行流程

双重调试让我们同时看到"模拟器如何工作"和"被模拟系统如何运行"，深入理解软件模拟硬件的本质。

4. 与 Lab2 的联系

Lab2 调试页表时也使用了双重 GDB，观察 `get_physical_address()` 函数如何软件模拟 MMU 的地址翻译。本次实验观察的是异常处理的模拟，两者本质相同：**QEMU 用软件实现硬件的复杂行为。**

5.3 实验感想

这次实验最大的收获是理解了"模拟器"的真正含义。之前只知道 QEMU 可以运行 RISC-V 程序，但不清楚具体如何实现。通过双重 GDB 调试，亲眼看到：

1. 用户程序执行 `ecall` 时，QEMU 的 `riscv_cpu_do_interrupt()` 被调用
2. 这个 C 函数精确地模拟了 RISC-V 硬件的行为：设置 `sepc`、`scause`、切换特权级
3. 内核执行 `sret` 时，`helper_sret()` 恢复状态并返回用户态

这种"软件模拟硬件"的思想非常优雅，也让我更深刻地理解了操作系统与硬件的交互方式。